

Video Game Programming: Online Game Programming

L18

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Online games

- Online games are (partially) played through a computer network
- They include:
 - Games for the Web, played using a browser
 - Games for different platforms that need a network to be played (at least in some modes, typically multiplayer ones)
- In addition, several games use a network to save players' profiles, handle leaderboards, get resources (e.g., DLC) or other data...



A game for the Web (FarmVille) and a multiplayer game (Fall Guys Ultimate Knockout)

Games for the Web

- Games for the Web used to rely on plugins such as Java and Flash
- Unity had its own plugin for Web games
- All these plugins are now no longer supported by browsers
- To support JavaScript-based Web games in an HTML 5 canvas, Khronos proposes the WebGL API, which runs without plugins
- Popular game engines have the possibility to build WebGL games
- To build WebGL games, Unity uses IL2CPP to convert .Net code in C++ and emscripten to generate WebAssembly files
- Next step is WebGPU

WebGL versions and features

- There are two main versions of WebGL (1.0 and 2.0), which roughly match OpenGL ES 2.0 and 3.0 functionalities
- With respect to other game platforms both versions cannot use threads, and have limitations in audio and networking features
- WebGL 1.0 also has graphical limitations (e.g., supports only forward rendering and gamma color space)
- WebGL 2.0 is supported by Firefox, Chrome, Edge (only Chromium-based), and Safari (only since v15)
- Compression, asset bundles, and stripping (being aware of issues) help reducing distribution size

AssetBundles

- Unity has a built-in system to manage online resources
- AssetBundle are archive files that contains non-code Assets such as models, textures, or entire scenes
- Unity can load AssetBundles at run time and then use their resources as built-in ones
- AssetBundles can be useful for DLCs, reducing initial install size, loading assets optimized for platform, reduce runtime memory
- AssetBundle also refers to the C# object to load Assets from an AssetBundle archive via code

AssetBundle creation and usage

- Assigning assets to bundles can be easily done using Inspector
- Assets can be optionally assigned to AssetBundle variants
- Custom editor code is needed to set the options to create the AssetBundle files
- AssetBundles can be loaded from memory, files (e.g., in StreamingAssets folder) or network (by means of a specific Unity Web Request and Download Handler)
- Then, specific or all assets can be loaded from the bundle and used in the game
- When the AssetBundle is no longer needed, it should be unloaded

AssetBundle grouping strategies

- Logical Entity Grouping bundles assets based on the function (e.g., all models and animations for a set of characters) and is ideal for DLCs
- Type Grouping assigns assets that are of similar type (e.g., audio tracks) and is useful for AssetBundles to be used by multiple platforms
- Concurrent Content Grouping bundles assets that will be loaded and used at the same time, so it is mostly used for scene bundles
- Each game can mix the strategies for different needs
- Additional tips:
 - splitting frequently updated objects from objects that rarely change
 - splitting in dedicated bundles assets who are dependencies for others

AssetBundle dependencies

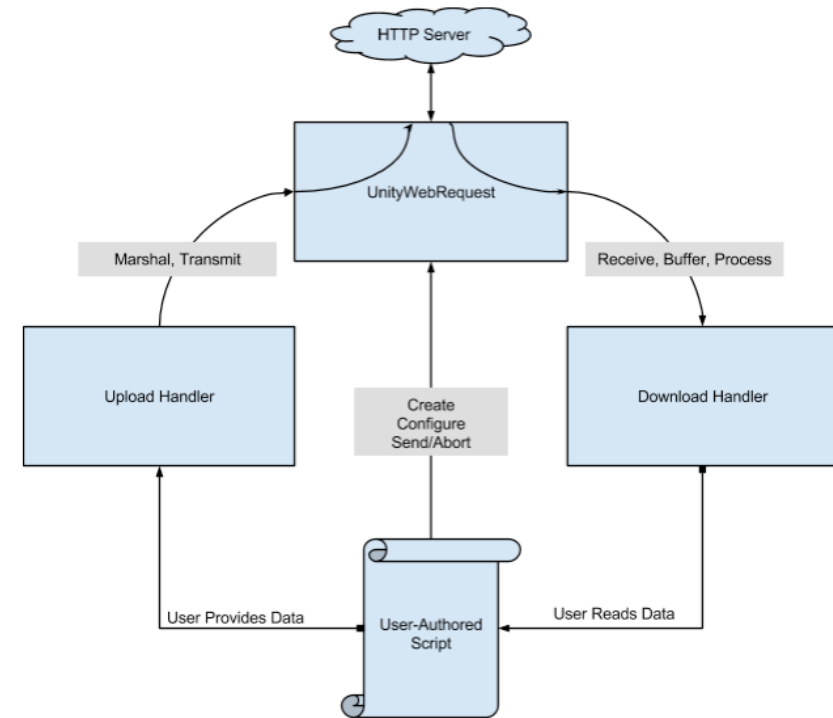
- Unity adds to bundles all needed dependencies which are not in a bundle
- This can cause duplication, e.g., if the same material is used in two bundles, both will include the required texture
- To avoid duplication, AssetBundles can have dependencies in other bundles, e.g., a texture bundle can be referred by material bundles
- It is fundamental that required bundles are already loaded before loading an asset from a dependent bundle
- Unity keep track of dependencies in a manifest file for each bundle and a manifest file for an overall manifest bundle
- Manifest can be used to find bundle dependencies to load at runtime

AssetBundle compression

- AssetBundles can be compressed with different algorithms depending on use case requirements:
 - No compression allows for fast load times, after long download times
 - LZMA compression (default for creation) results in smallest file size, but requires longest load times, because the entire bundle is decompressed before use
 - Chunk-based LZ4 compression (default for caching) results in larger file sizes than LZMA, but does not require that entire bundle is decompressed on load, because it can load the AssetBundle in chunks
- Using chunk-based compression has comparable load times to uncompressed bundles with the benefit of reduced size on disk

Online resources

- Network functions concern:
 - Download of resources on demand, from getting a text or image file to managing bundles with several assets
 - Access to remote databases, by getting and sending data (e.g., in json from / to a PHP page)
 - Use of social APIs, by integrating plugins with specific functions
- Game engines can offer low-level and high-level network APIs, as well as wrappers for social APIs



UnityWebRequest low- and high-level API

Social APIs

- Google Play Games plugin for Unity (<https://github.com/playgameservices/play-games-plugin-for-unity>) allows to use Google Play leaderboards and trophies
- Apple GameKit plugin for Unity (<https://github.com/apple/unityplugins/tree/main>) allows to integrate GameKit features such as leaderboards, achievements, and match making